MyScoutee
OFFICIAL PRODUCT DOCUMENTATION

ARCHITECTURE & DEPLOYMENT

MyScoutee Microservices Guide

One codebase, configurable service assemblies, production-style images, and an explicitly controlled KVM laboratory.

DOCUMENT ID MSC-MS-001	**DOCUMENT VERSION** 0.1.2
APPLICABLE SOFTWARE VERSION Canonical backend and eight-service assembly configuration	**STATUS** ● Implementation preview
RELEASED 2026-10-15	**LANGUAGE** EN

Document Control

Scope and authority

Document owner	MyScoutee — backend architecture and deployment
Audience	Contributors and operators, including readers new to virtualization, Kubernetes and metrics
Source of truth	`microservices/assemblies.json`, versioned lifecycle scripts, canonical domain sources and recorded verification
Review triggers	Domain regrouping, new dependencies, transport/security changes, runtime versions, resource budgets or qualification results

Version History

Version	Date	Change
0.1.2	2026-10-15	Central Feedback ownership, 22-domain composition, combined page contract and domain transport boundaries.
0.1.1	2026-10-10	Eight-service separation, platform deployment capabilities, retained minimal-template models, KVM deployment and local release simulator.
0.1.0	2026-10-09	Service assemblies, native recovery journals, replica-write checks, bounded read-load tooling, canonical data imports and observability. Includes the KVM/Kubernetes/Helm layer map, operation and cleanup procedures.

> **Deployment scope**
> This guide describes the development laboratory and its operating procedures. Production adoption requires qualification against the intended workload, infrastructure and recovery requirements. Startup, schema compilation and a successful metrics scrape do not establish complete business-flow correctness.

Contents

Architecture and ownership

Build and artifact boundaries

Communication and execution context

Developer-host installation and VM lifecycle

Kubernetes and live runtime inspection

Prometheus, Grafana and localhost access

Application and stress-test completion gates

Stop, destroy and host cleanup

Rules for future changes

Verification and known limits

1. Architecture and Ownership

Business source remains in `server/projects/profile/src/main/java/com/raxim/myscoutee/domain`. The legacy `projects/profile` name is intentional. The monolith and service builds compile the same source. The `microservices/` shell owns composition, entry points, transport and deployment. It can later move into a separate repository mounted as a submodule.

Canonical domains own business behavior and configuration; the microservices shell owns assembly, transport and deployment. The monolith remains buildable without the shell. One shared `CoreApplication` source is compiled into each selected artifact.

1.1 Read the system in layers

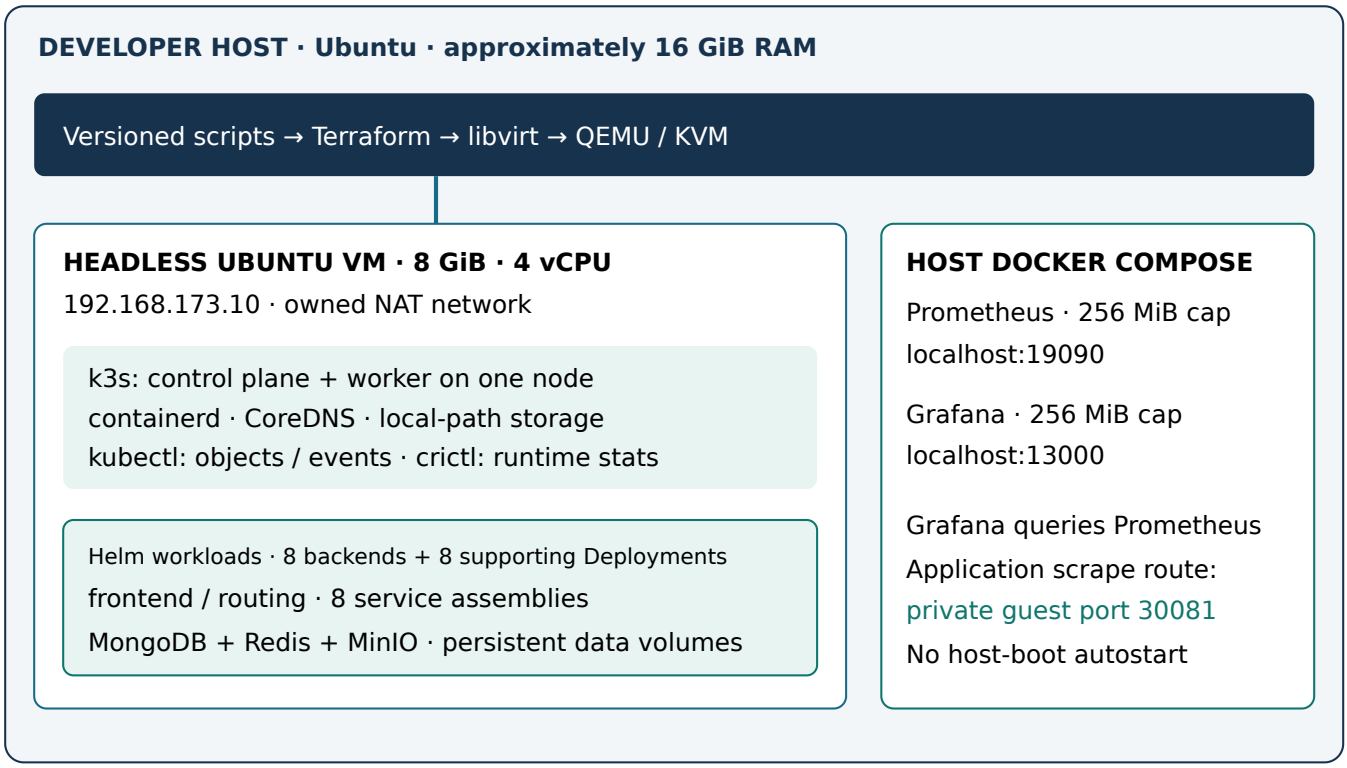
A source domain, a service image, a container and a virtual machine are different things. A domain groups Java responsibilities. An assembly selects domains and produces one executable image. A running image becomes a container. Kubernetes places containers in pods inside the Ubuntu guest. KVM runs that guest on the developer's physical computer. Several independent application processes therefore share one guest, instead of requiring one VM per service.

The tools used here and the question each answers

Layer / tool	Responsibility in this repository
Gradle + Docker image build	Which canonical source and runtime dependencies go into each immutable application package?
KVM + QEMU	How does the headless Ubuntu guest execute with virtual CPU, memory, disks and a network interface? KVM supplies Linux hardware virtualization; QEMU supplies the machine and devices.
libvirt + virsh	Which guest, storage pool and virtual network exist, and are they running? libvirt manages QEMU; virsh is its command-line client.
Terraform	Which of those infrastructure resources should exist, and which existing resource does each declaration own?
cloud-init + systemd	How is a fresh guest configured, and how does its k3s service start when that guest boots?
k3s / Kubernetes + containerd	Which pods should run inside the guest? Kubernetes reconciles workloads; containerd actually runs their containers.
Helm	Which versioned templates and values describe the application Deployments, Services, configuration, credentials and volume claims?
Prometheus	What numerical measurements have the configured targets exposed over time?
Grafana	How can an operator query and compare those measurements in dashboards?

Follow one change through these layers: changing a Java domain needs a new assembly image; changing which domains share a process changes `assemblies.json`; changing VM memory changes infrastructure. A Grafana panel changes presentation of measurements. Diagnose and edit the layer that owns the problem before restarting surrounding layers.

1.2 Current physical placement



The workload box contains all eight backends and eight supporting Deployments. Host Docker builds images and runs monitoring; the guest uses containerd, so host `docker ps` does not list guest pods. Monitoring lives beside the VM, outside Kubernetes. Its application scrape targets need an explicit private network connection; simply installing Kubernetes does not provide that connection.

1.3 Application composition

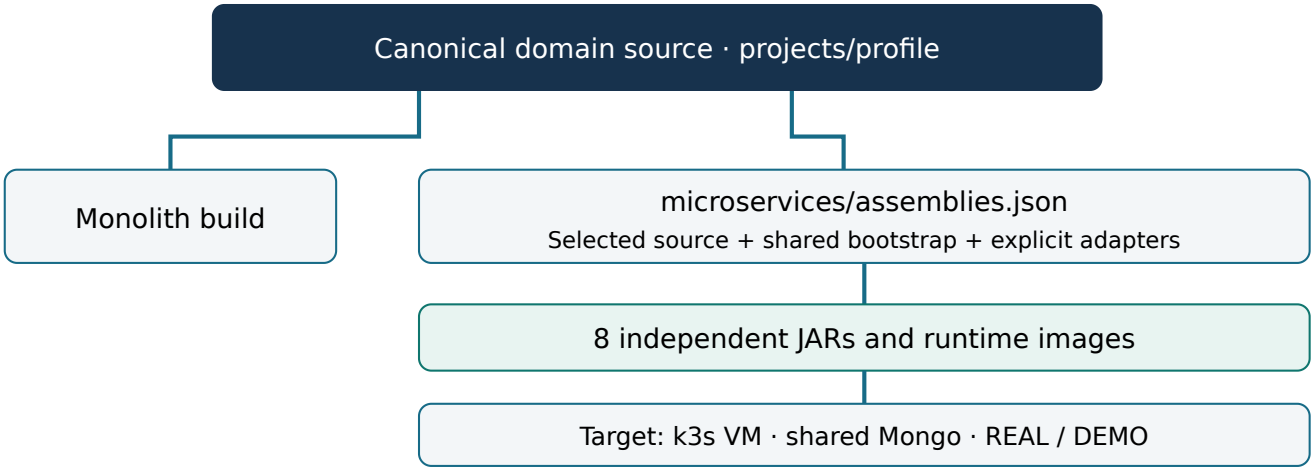
Current source grouping from assemblies.json

Service	Domains	Heap / limit
identity	accounts, access, administration, platform, integrations	512 / 1024 MiB
community	groups, community, events, feedback, campaigns, content	512 / 1024 MiB
communication	chat, notifications, email, localization	384 / 768 MiB
assets	assets, media	384 / 768 MiB
payments	payments	384 / 768 MiB
affinity	affinity	384 / 768 MiB
help	help	256 / 512 MiB

Service	Domains	Heap / limit
network	operator, registry	384 / 768 MiB

> **Platform and product boundaries**
> Generic bootstrap, configuration, TLS and updates belong to platform. Help remains a separate service; the existing content domain remains in community. Operator/Registry form the MyScoutee-specific network assembly and are excluded from the minimal template. Development and KVM must run serially. The template retains the required canonical models unchanged, including identity/session, profile, group/workspace, chat, content/consent, localization and geographic location foundations; screens may be omitted. The platform owner alone receives its persistent configuration key. The lab reuses Compose's release simulator and receives only its public TLS CA, with no GitHub fallback. An empty local catalog proves discovery only.

The current source defines eight backend deployments for 22 domains. The mapping is configuration: splitting a group adds an assembly and reassigns its domains. Build project creation, precise Spring scans, Dockerfiles, Helm workloads and monitoring targets follow that map. For implemented boundaries, `transport/bindings.json` drives generated imports and Helm endpoint addresses from domain ownership, so a regrouped domain receives the matching network destination. A new network boundary still requires a reviewed transport adapter. Helm applies the listed memory limits. The frontend has one replica; the requested scaling experiment must use a working backend service.



The laboratory uses one shared MongoDB replica set, with two members and distinct REAL and DEMO databases. Collection and workspace ownership remain domain concerns. Sharing Mongo does not make cross-process calls atomic. Bounded connection pools are part of deployment, not a reason to create another Mongo per service.

2. Build and Artifact Boundaries

Commands below run from the backend repository root. Check the current host first. If a dev watcher already owns compilation, follow the repository runtime rule; never run a competing build.

```
microservices/tools/lab.sh preflight
microservices/tools/lab.sh jars
microservices/tools/lab.sh images
```

`build` combines the last two steps. The wrapper uses serial Gradle execution and the backend's existing wrapper, dependency catalog and security overrides. It does not reuse the monolith's compiled output.

Output	Purpose
`build/<service>/libs/`	Independent executable JAR
`build/<service>/generated/`	Precise component/repository configuration and production resources
`build/deployment/`	Per-service Dockerfile and monitoring targets
`build/reports/assemblies.json`	JAR hashes, sizes, dependency and package ownership evidence

The audit rejects foreign implementation packages, duplicate application classes, the monolith entry point and development resources. API/DTO contracts are compiled once into a shared library. Existing Java projections are not automatically safe wire schemas.

Images use the canonical production YAML and Java 25 JRE, separate dependency/application layers, user 10001, bounded heaps and fail-fast OOM handling. Only the assets runtime carries ImageMagick/WebP tools. No source mounts, sync containers, watchers, DevTools or build tools belong in the measured runtime.

To build from a future separate shell checkout, set `MYSCOUTEE_BACKEND_SOURCE` to the compatible backend source checkout. Preserve the canonical source path; copying domain code creates another owner and is unnecessary.

2.1 Image versions and reuse

```
# Explicit release actions, only after relevant source changes:
microservices/tools/lab.sh frontend-image
python3 microservices/tools/workloads.py lock-images

# Ordinary deployment: import missing versions; never build implicitly.
python3 microservices/tools/workloads.py up
```

`deploy/images.lock.json` records full Docker image configuration IDs, content fingerprints and version references. A reference such as `myscoutee-service-assets:0.1.0-<hash>` is checked against its full ID before reuse. The host and guest have different image stores. `import-images` streams only missing locked images from host Docker to guest containerd over the existing pinned SSH connection. Repeated deployment reuses the locked guest cache without rebuilding. The inventory now contains eight backend and nine supporting images.

Kubernetes uses `IfNotPresent`. This single-node lab does not need an additional registry container. Custom tags are local artifacts, not published Docker Hub images: a new machine must receive the matching artifact or build it explicitly. A future registry can distribute published versions; cache presence and exact identity must still be verified.

3. Communication and Execution Context

The first implemented boundary retains `localization.api.TranslationBundles`. The canonical `I18nService` owns bundle selection, language fallback, mode-specific caches and version semantics. The shell supplies a typed HTTP adapter and a thin endpoint.

```
GET /_internal/v1/localization/bundle
Authorization: Bearer <dedicated localization-read secret>
X-MyScoutee-Session-Mode: REAL | DEMO
```

The checked-in contract is `microservices/transport/contracts/localization/localization-v1.openapi.json`. Optional query parameters are `lang`, `acceptLanguage` and `version`. The response reuses `I18nBundleResponseDto`; a current version returns `data: null`, preserving the canonical distinction from an empty map.

Boundary	Behavior
Authority	Dedicated read credential; no access to arbitrary domain routes or management endpoints
Execution scope	Mandatory REAL/DEMO header after authentication; context restored after success and failure
Limits	1-second connect / 3-second total request defaults; 2 MiB response maximum; eight concurrent reads, no queue
Failure	No redirects, automatic retries or local fallback; bounded error codes without persistence details
Metrics	Receiver HTTP metrics and client `myscoutee.remote.requests` with fixed operation/outcome labels

The generated assembly imports the server configuration in the localization owner. An assembly importing `TranslationBundles` without owning localization receives the client configuration. With the current grouping, identity is a remote consumer; email remains local within communication.

```
myscoutee.remote.localization.url=https://communication.internal
myscoutee.remote.localization.read-token=<private generated secret>
myscoutee.remote.localization.allow-insecure-http=false
```

Supply the read secret privately to the owner and intended consumers; generate at least 256 random bits. Plain HTTP requires explicit isolated-lab opt-in. Production requires a trusted TLS path and credential custody/rotation. A read credential for global translations is not proof of delegated account or workspace authority.

3.1 Focused media-policy contracts

The identity assembly also consumes the existing `GroupMediaVisibility` and `CommunityMediaVisibility` APIs from the community assembly, plus `AssetMediaVisibility` from the assets assembly. Event, feed and chat media reads use the same shared bounded transport

and retain their canonical owners. Each operation returns only the metadata that the canonical media policy reads. The APIs remain in domain `api` packages; concrete transfer records belong to domain `dto` packages. The shell owns HTTP adapters and composition.

Operation	Response	Contract
`GET /_internal/v1/groups/media-visibility?id=…`	`GroupMediaVisibilityDto`: lifecycle status	`groups-media-v1.openapi.json`
`POST /_internal/v1/community/media/announcement`	`AnnouncementMediaVisibilityDto`: community ID and status	`community-media-v1.openapi.json`
`POST /_internal/v1/community/media/offering`	`OfferingMediaVisibilityDto`: base-group ID and status	Same contract, separate operation
`POST /_internal/v1/assets/media-visibility`	`AssetMediaVisibilityDto`: owner ID, visibility, lifecycle and moderation status	`assets-media-v1.openapi.json`
`POST /_internal/v1/events/media-visibility`	`EventMediaVisibilityDto`: visibility and member projection	`events-media-v1.openapi.json`
`GET /_internal/v1/events/media-membership`	`CommunityMediaMembershipDto`: accepted community member role	Same contract, separate operation
`POST /_internal/v1/content/media-visibility`	`FeedPostMediaVisibilityDto`: moderation status	`content-media-v1.openapi.json`
`POST /_internal/v1/chat/media-visibility`	List of `ChatMessageMediaVisibilityDto`: chat ID and deletion timestamp	`chat-media-v1.openapi.json`

The POST operations are read queries containing `{"urls":["…"]}`. Existing media checks use URL variants that can exceed request-line limits; one bounded JSON request preserves that batching. Limits are 32 URLs, a 1-MiB body including chunked transfers and a 4-KiB response for small projections. Event member projections and chat-reference lists allow up to 2 MiB. Chat absence is `200 []`; all matching references remain available to the existing forwarded-attachment access policy. Each boundary has its own private read credential. A verified `404 NOT_FOUND` retains canonical absence; a proxy 404, timeout or owner failure stays an error. These service capabilities supply facts to media authorization; they do not authenticate a user or grant access to a file.

The existing localization transport supplies the shared timeout, concurrency, credential and mode-context implementation. The assembly audit also verifies actual client/server imports in the compiled configuration, including consumers that use fully qualified Java names.

3.2 Focused profile identity reads

`accounts.api.ProfileIdentities` returns `ProfileIdentityDto` containing only user ID, account ID and workspace ID. Separate operations preserve the canonical first-profile lookup, non-deleted existence check and workspace user-ID list. The canonical `ProfileDirectoryService` owns these queries; the shell transports them through `profile-identities-v1.openapi.json`. Payment administration, affiliate validation and refund notification reuse this port. Their existing authorization and notification rules remain in their canonical owners.

This is a use-case contract, not a serialized profile document. Other profile consumers still need reviewed contracts for the facts they use. A service read credential does not delegate the caller's administrative role or workspace authority.

The binding inventory associates each port with its owning domain, explicit Spring configurations, configuration prefix and private credential name. The build selects local owner or remote client; the workload renderer resolves the owner's Kubernetes Service name. Credential migration adds missing secrets atomically without replacing existing values.

3.3 Chat directory reads

`chat-directory-v1.openapi.json` transports the existing owner/chat lookup, all directory views for a chat, unread version and support counts. `ChatDirectoryEntryDto` carries only the existing directory interface fields, including member IDs and support status. Chat messages, persistence IDs and other document fields are excluded. Tests wrap the canonical directory service with repository/Mongo fixtures, preserving null versus empty values, REAL/DEMO scope and aggregate-count semantics.

3.4 Additional focused reads

`NotificationCatalog` exposes the existing template and unread, failed-job and outbox counts through `notification-catalog-v1.openapi.json`. `RecomputeQueue` exposes the canonical affinity queue count through `recompute-queue-v1.openapi.json`. Neither read capability grants write access.

`CaseParticipants` preserves case conversation views, active participant account IDs and visible-case counts through `case-participants-v1.openapi.json`. Its bounded POST count query contains workspace, account and managed-group IDs; canonical membership filtering remains in the owner. `CampaignMediaVisibility` narrows the campaign catalog's existing media lookup to a DTO containing only workspace and status, documented by `campaigns-media-v1.openapi.json`.

Central Feedback ownership

As of 15 October 2026, Feedback is a separate domain inside the community deployment. It owns `feedbackStates`, `feedbackAnswers` and `feedbackEvaluations`, including evaluation submission, persisted counter transitions and combined pagination. Events and Community call its API; sharing a Java process does not permit access to its repositories or documents.

The Feedback screen loads one complete page through one service operation. Its response includes event and completed-case entries, display metadata, totals and bucket counts. HTTP uses one page request; frontend local mode uses the same contract and one simulated request delay. Local evaluation records are stored separately from Community cases. Displayed answers remain labelled choices; their internal numeric encoding is not a card score.

The domain exposes `FeedbackDirectory`, `FeedbackPreparation`, `FeedbackEligibility`, `FeedbackMeasurement` and `FeedbackPrivacy` transport boundaries. Consumers use these owner-defined contracts in both the monolith and service assemblies; sharing a process or database does not grant access to Feedback persistence.

3.5 Membership reads and policy facts

`member-directory-v1.openapi.json` transports the existing membership lookup/list, batch, page and VIP-pricing operations. The detached member DTO excludes attendance, counters and invitation audit. Queries are bounded to 4096 owner IDs and 1 MiB; pages accept 1–1000 entries and responses at most 2 MiB. Canonical VIP eligibility remains owner-side. Errors never turn into absent membership or free pricing.

`EventMembershipFacts` and `AssetMembershipFacts` narrow their existing ownership APIs to the fields read by `MemberPolicyFacts`. Their DTOs contain five event fields and four asset fields. Local services retain the original queries through API inheritance; the shell adds explicit HTTP transport. Workspace filtering, lifecycle and role policy remain unchanged. Existing canonical access-control/resource tests cover those rules; HTTP fixtures cover the detached projections and session-mode boundary.

`GroupDirectory` retains its nine existing directory operations, including owner-computed lifecycle and moderation facts. Its DTO implements the existing read view; group policy derivation is not repeated in the HTTP client. `CommunityMembershipReads` narrows `CommunityMemberships` to the two reads used by account workspace selection. Membership registration and transitions stay owner-local.

`EdgeDirectory` exposes the two existing affinity directory reads. `AffinityEdgeDto` contains exactly the seven interface fields; private matching evidence is excluded. The batch accepts at most 4096 identifiers and 1 MiB, and responses are bounded to 2 MiB. Owner failure is reported as an error, never an empty network.

`RatingDirectory` retains four canonical reads and the existing 23-field rating view. Supplemental actor snapshots are excluded. Home-game exclusion projections remain sparse, and positive campaign ratings use the original owner query; HTTP tests cover those semantics, decimal scores, null fields and owner failures.

`ChatReview` transports reported-message and report-evidence reads while retaining canonical membership/author checks and message order. `EventHomeReads` narrows the home activity/report queries to thirteen consumed event fields; `FeedbackDirectory` preserves the existing target-user query and answer view. The unused asset ownership dependency was removed from the home consumer.

> **Remaining communication work**
>
> Actor/workspace validation, other domain adapters and public authentication remain open. The nginx routing table exists, but routing cannot supply absent application bindings. SDK handles, servlet objects and callback interfaces cannot be serialized into generic RPC. Payment finalization must reconcile an unknown remote outcome before compensation; a timeout alone does not justify a refund. Reuse the existing payment coordinator and integration outbox owners.

3.6 Discovery, gateway and failure recovery

A Kubernetes Service supplies a stable address; CoreDNS resolves its service name to that address. This is service discovery. The typed adapter still serializes a request, carries REAL/DEMO and workspace context, checks authority, imposes limits and validates the reply. OpenAPI describes this wire contract. DNS does not implement Java calls, and a service mesh would not make a multi-owner mutation atomic. No mesh is installed in this lab.

The frontend Nginx also routes public API paths to their owning assembly. This is the current gateway layer. Signed short-lived actor delegation carries the existing authenticated identity across internal calls; private capability credentials restrict which producer can invoke each owner. Keep business policies in their existing domains. An empty-body health response is neither an authorization decision nor a completed business operation.

Recovery belongs to the business owner that accepted the command. Contact access now records an immutable accounts-owned command before the chat write, retains its receipt, applies version-guarded account effects and uses the existing notification outbox with a stable operation key. Profile status propagation retains its pending marker with the profile save and uses the existing affinity queue. Integration image retries use deterministic keys for the same owner/entity/content; uncertain resource creation must not delete a possibly referenced image. A successful payment fulfillment followed by a failed receipt save remains pending for gateway redelivery, never an automatic refund.

The accumulated standard, embedded recovery and adapter checks pass for these additions. Notification queue counts are a write-side projection refreshed by its existing worker, so they converge after a crash and may briefly lag concurrent writes. Provider push delivery retains its existing at-least-once behavior; an external provider without an idempotent receipt cannot promise exactly-once delivery. This does not require deploying a new broker or a general saga engine.

4. Developer-Host Installation and VM Lifecycle

4.1 What the host owns and what the guest owns

Headless means there is no desktop installed in the guest. Its Linux services are inspected over authenticated SSH. The physical host keeps the virtualization packages; guest packages, Kubernetes data and eventual application volumes live on the VM disk. Destroying the VM removes that guest data, while host package removal is a separate tracked operation.

The inspected host is Ubuntu 22.04 on x86-64 with KVM support and approximately 16 GiB physical memory. The host installer intentionally supports this inspected platform. Capacity and route checks run again before startup; historical free memory is not a startup guarantee.

```
bash microservices/tools/setup-host.sh
python3 microservices/tools/vm.py preflight
python3 microservices/tools/vm.py start
python3 microservices/tools/vm.py status
python3 microservices/tools/vm.py observe
```

Run setup in your terminal. `sudo` handles password entry; no script stores or reads the password. Host setup records package versions, existing packages, service ownership and added group membership before changing them. It installs headless QEMU/libvirt dependencies and starts the

host services for the current session only.

4.2 Desired state, recorded identity and first boot

Terraform configuration describes the desired resources. Its provider translates that configuration into libvirt operations. `plan` compares declarations, recorded identities and live resources; `apply` performs the proposed changes. Here `vm.py start` runs both with additional ownership and safety checks. Re-running an unchanged start should report *No changes*; it must not create a second VM.

The seven managed objects are the VM, NAT network, directory storage pool, base Ubuntu volume, writable system volume, cloud-init disk generation and its uploaded volume. The system disk stores changes over the verified base image. Its 40 GiB capacity is sparse: this is a growth limit, not 40 GiB allocated immediately.

The cloud-init ISO supplies the initial user, SSH public key, packages and k3s configuration to a fresh guest. Cloud-init is initial provisioning, not a continuous configuration reconciler. The guest installer can reconcile its pinned k3s files, but changing cloud-init input on an existing VM is not automatically a safe live upgrade. The startup wrapper rejects a plan that would replace persistent resources; review such changes explicitly.

Terraform is the declarative owner of the VM, storage pool, volumes and NAT network. Its private state records resource identities so later runs can reconcile the same resources and remove only those resources. Cloud-init installs the guest dependencies and versioned k3s service. Terraform does not itself uninstall host packages.

4.3 Resource and access boundaries

Item	Default
Guest	Headless Ubuntu, 4 vCPU, 8192 MiB RAM, 40 GiB sparse system disk
Network	Owned NAT `192.168.173.0/24`; guest `192.168.173.10`
Host reserve	At least 2 GiB available beyond the stopped VM's requested memory
Tool/artifact pins	Version/date and SHA-256 in `infra/versions.json`; provider lock file in Git
Secrets and state	`~/.local/state/myscoutee-microservices/msc121/`; mode 0700
Root inventory	`/var/lib/myscoutee-msc121-host/`; root-owned, mode 0700

The default host reserve remains 2048 MiB. An explicit `vm.py start --host-reserve-mib 1536` applies a smaller reserve to that invocation only, as authorized for this host. It does not shrink the 8192-MiB guest or persist a weaker default. The monolith E2E stack and KVM lab must run serially; stop the VM completely before `bash microservices/tools/check-current-monolith.sh`. That script builds the current backend, reuses the canonical disposable E2E stack and scenarios, records private evidence, and stops its stack on exit.

The VM, lab network, pool and newly installed host daemons must not autostart after a host reboot. Guest k3s may start when the VM is explicitly started. The checks include libvirt sockets and guest save/resume, not only the VM autostart flag. No VMware or desktop guest is involved.

Startup rejects conflicting subnets, lost resource identity, insufficient memory and a plan that would replace persistent disks. Failed provisioning retains state for diagnosis. Ubuntu AppArmor remains enforcing: explicit disk and backing-image paths belong in the domain definition.

Guest SSH identity is obtained through the authenticated libvirt guest-agent channel and stored in the private known-hosts file. Guest access does not alter normal SSH configuration. Kubeconfig is copied to private lab state, never to the user's default Kubernetes configuration.

A kubeconfig contains the API address and credentials used by Kubernetes clients. Treat it as private access material, just like the guest SSH key. Terraform state is also private operational state; it is not a disposable cache. Losing it breaks the safe association between declarations and existing resources.

The eight source-defined application memory caps total 6400 MiB; their Java heaps total 3200 MiB. These are per-container upper bounds, not committed RAM. The 8192 MiB guest also needs room for native JVM memory within each cap, guest OS, k3s, data services and frontend/routing. Available memory is checked before replica/load runs. Host monitoring adds up to 512 MiB outside the guest, plus virtualization overhead. These budgets need measurement under load; a successful VM preflight does not prove that the completed application fits.

4.4 A normal session

1. Install once with `setup-host.sh`, entering sudo credentials in your terminal. Builds and VM startup must respect the existing dev-owner rule.
2. After a host reboot, explicitly run `sudo python3 microservices/tools/host.py start`. This starts the installed host services for this session.
3. Run `vm.py start`. Expect a longer first run for verified downloads and guest setup; later starts retain guest identity and disks.
4. Wait for cloud-init completion and the k3s API readiness message. Then inspect the node and workloads as described next; API readiness alone is not application readiness.
5. Start monitoring separately with `lab.sh monitoring-up`. End the session with `vm.py stop` and `lab.sh monitoring-stop` to retain data.

5. Kubernetes and Live Runtime Inspection

5.1 The Kubernetes objects used in this lab

k3s is the Kubernetes distribution installed inside the guest. One server node runs both the control plane, which stores desired state and coordinates work, and the worker components, which execute containers. The default embedded SQLite database stores Kubernetes state; it is separate from the application's MongoDB. This one-VM topology provides a realistic process/network boundary, but no protection against losing that VM or physical host.

Term	Meaning here
Node	The Ubuntu VM registered as `myscoutee-msc121`. Its Ready condition describes the worker's ability to accept work.
Pod / container	A pod is Kubernetes' scheduling unit. A container is the running image inside it. For the planned application, each assembly starts its own JVM container; pod identities may change when replaced.

Term	Meaning here
Deployment	A desired set of replaceable pod replicas and their image/configuration. Each of the eight backend Deployments normally has one replica. The bounded scale experiment temporarily gives assets two replicas and restores one afterwards.
Service / CoreDNS	A Service gives selected pods a stable cluster address. CoreDNS resolves service names inside the cluster. A Service does not automatically expose its pods at the host's localhost.
Namespace	A grouping of cluster objects. `kube-system` contains system components. This is not a MyScoutee tenant, workspace or REAL/DEMO database boundary.
Local-path storage	The bundled provisioner can create persistent storage on the guest disk. A volume can outlive a pod, but destroying this VM destroys its local backing data.

Traefik, ServiceLB and metrics-server are deliberately disabled in this configuration. There is currently no automatic application ingress or load balancer, and `kubectl top` is unavailable. Use containerd statistics through `k3s crictl stats` for the requested docker-stats-like view. No additional dashboard server is needed for that view.

Supporting services and production parity

Component	Current state and purpose
MongoDB	Two members, separate 4-GiB persistent volumes, shared REAL/DEMO databases. Primary reads; majority writes require both members.
Redis	One running pod with bounded ephemeral cache; existing cache and realtime coordination.
MinIO	One running S3-compatible object-store pod with a 1-GiB persistent volume and 384-MiB memory limit. Holds packaged media and uploads.
nginx / frontend	One production frontend pod with API routing; backend readiness is separate.
Prometheus / Grafana	Host containers with a verified private metrics gateway into the guest; application samples require ready backends. See the monitoring chapter for URLs and login.
STUN	Running with a 64-MiB limit. Canonical STUN-only coturn; VM NodePort 30478 supports UDP and TCP. No TURN relay.
MCP server	Running with a 128-MiB limit. Canonical prod image uses the service gateway; public and private MCP routes preserve streaming/authentication.
DEMO registry / payment simulator	Both running with 128-MiB limits and separate 1-GiB PVCs. Canonical start-demo registry and DEMO payment provider; browser and callback routes are configured. Only sampled business flows are qualified; REAL outbound registry stays disabled.

Component	Current state and purpose
Firebase / FCM, SendGrid	Disabled external providers; these are not local container services.

All listed supporting production containers are deployed. Healthchecks alone do not prove end-to-end business integration. No extra Kafka, RabbitMQ or SQL database is required by the configuration being packaged.

MinIO uses the canonical production Dockerfile and seed/start scripts, with no duplicated storage implementation. Its private Service exposes S3 at `http://minio:9000` and the console at `http://minio:9001` inside the cluster. The existing backend prod configuration receives that endpoint, region `eu-central-1` and bucket `myscoutee-media`. There is no automatic host console port.

```
bash microservices/tools/lab.sh minio-image
python3 microservices/tools/workloads.py lock-images
python3 microservices/tools/workloads.py frontend-up
python3 microservices/tools/check-minio.py --seed --restart
```

The explicit image build is cached; workload startup only imports missing immutable images. `frontend-up` is the explicit frontend-only diagnostic mode; normal `up` starts the complete locked workload set. MinIO readiness waits for seed completion. It runs as user 10001 with a read-only root filesystem, persistent `/data` and ephemeral `/tmp`. Workload stop retains the volume; destroying the owned VM removes its local backing data.

The generated private password stays in the existing protected workload state and Kubernetes Secrets. Access ID `lab-media` and that password configure both bootstrap and the canonical S3 client, matching the current prod credential pattern. Restricted separate service credentials remain a production concern. Do not copy the password into this guide or Git.

The object-storage check exercises write/read, pod replacement, persistence through canonical reseeding and probe-object cleanup. Existing media keys remain untouched. This storage check does not establish correctness of the full backend media flow.

```
bash microservices/tools/lab.sh support-images
python3 microservices/tools/workloads.py lock-images
python3 microservices/tools/workloads.py frontend-up
python3 microservices/tools/check-support.py
```

The support image build reuses the canonical sibling Dockerfiles serially and never runs on workload startup. The versioned support check covers healthchecks, host-to-VM STUN UDP/TCP Binding Responses, the MCP OAuth challenge, simulator browser assets and denial of public access to internal/management routes. MCP is available at `http://localhost:18080/mcp`; its private endpoint requires OAuth. Simulator browser surfaces use the same localhost origin and retain their normal access checks.

The gateway uses full Kubernetes DNS names because nginx's dynamic resolver does not apply pod search suffixes. Simulator provider/control APIs stay private; public browser paths match the prod allowlist. Generated test API/webhook/POS credentials are shared privately with payments. DEMO registry server keys/data and client identity have separate retained PVCs. Client storage first binds when the owning backend starts.

This host-accessible lab advertises `stun:192.168.173.10:30478`; public deployment requires an address reachable by its clients. The coturn binary is invoked directly in both Helm and the shared Compose definition because the image wrapper discards the `-n` option. Observed idle memory was about 3 MiB each for MCP and simulator, 11 MiB for registry, and 5 MiB for STUN. These are not stress measurements.

Canonical data initialization

```
python3 microservices/tools/workloads.py frontend-up
python3 microservices/tools/seed-mongo.py
python3 microservices/tools/seed-mongo.py --restart
python3 microservices/tools/check-minio.py --seed
python3 microservices/tools/check-support.py --data --restart-data-services
```

Shared two-member Mongo replica set

`deploy/runtime.json` enables `mongoReplicaSet`. Both members reuse the locked Mongo image, with a 768-MiB memory limit and 256-MiB WiredTiger cache per member. The configured Mongo CPU ceiling is three cores; it shares the existing four guest CPUs with the other pods. Primary reads preserve consistency and do not distribute read load to the secondary automatically.

```
python3 microservices/tools/mongo_replica.py configure
python3 microservices/tools/mongo_replica.py status
```

The conversion script stops backend writers, creates and validates a private authenticated archive, retains the original PVC, deploys the second member, initializes only the expected replica set, compares collection counts, and verifies a majority write by reading the secondary before deleting the probe. It then restores the backends. The repeat run verifies the receipt and configuration without another conversion or shutdown. Keys, backups and credentials stay outside Git. Normal stop includes both members; destroying the owned VM also deletes its backups and volumes.

Each Mongo member has a headless Service (`clusterIP: None`) with `publishNotReadyAddresses: true`. CoreDNS therefore publishes its pod address before readiness. The existing init container waits until that member name resolves to its own pod IP before starting Mongo. This avoids a simultaneous-restart bootstrap cycle where Mongo cannot identify itself through an unready Service. The workload reconciler migrates only the two owned member Services while backend writers are stopped; volumes are retained.

Two voting members need both for majority writes. This demonstrates replication, not continued writes after a member or host fails. No arbiter or secondary-read policy is added. Never disable the replica-set flag as an implicit data rollback.

Mongo uses the existing production importer and its collection allowlist from `server/.env`. The canonical `myscoutee_db` and `demo_db` sources map to the isolated `msc121_real` and `msc121_demo` databases. Bootstrap requires empty databases and stopped backends. Its durable content fingerprint permits identical interrupted work to resume and makes repeated completed bootstrap a no-op. A changed seed or namespace is rejected; this command does not update an existing application's data.

`seed-mongo.py --verify` checks the completed bootstrap without importing. `--restart` additionally replaces only the Mongo pod and compares every collection hash/count, index definition and bootstrap marker. Both verification modes require stopped backends.

The canonical bootstrap separates REAL and DEMO identities and imports articles, translation messages and policy definitions through their existing seed owners. The MinIO audit compares source keys and SHA-256 hashes with packaged files and actual S3 objects. Application uploads are preserved.

The registry's native DEMO seed and rolling QMAU refresh run through `start-demo`. The payment simulator initializes its SQLite state and built-in DEMO cards with the same code used by prod. Lab keys and persistent state are isolated. The restart check verifies retained registry seed/signing identity and simulator configuration/credential. Kubernetes volume ownership uses `OnRootMismatch`; a non-root init container restores owner-only `0600` permissions on the existing registry signing key and VOPRF keyring. It neither reads nor regenerates those keys. Redis is cache, and MCP/STUN need no separate business-data import. The frontend remains the canonical packaged bundle. Backend cache loaders and index initializers stay in their owning domains.

5.2 Open a live view or the guest shell

```
# On the host, from the repository root:
python3 microservices/tools/vm.py observe
python3 microservices/tools/vm.py observe --once
python3 microservices/tools/vm.py console
```

`open-terminals.sh runtime` opens the versioned monitor and guest shell; repeated calls do not duplicate an open view. The monitor separates host/VM state, Kubernetes workload readiness and host Docker monitoring. It rewrites one terminal screen with a one-second target interval; slow reads never overlap. Rows are clipped to terminal size, input echo is disabled while observing, and the original terminal mode is restored on exit. Ctrl+C closes only this view. `console` opens the existing pinned SSH connection; type `exit` to leave the VM running.

```
# Inside the guest console; read-only inspection:
sudo k3s kubectl get nodes -o wide
sudo k3s kubectl get deployments,pods,services -A
sudo k3s kubectl get events -A --sort-by=.lastTimestamp
sudo k3s crictl stats
free -m
sudo systemctl status k3s --no-pager
sudo cloud-init status --long
```

Read the output in order: a Ready node confirms the worker; desired/ready Deployment counts show reconciliation; each pod's Ready column counts ready containers. A Running pod can still be unready. Repeated restarts require events and previous container logs. If a future split introduces a missing binding, use the standalone context probe before rebuilding images. MongoDB, Redis, MinIO and frontend readiness do not make those backend services healthy.

For a failing pod, use the actual namespace and pod name from the listing with `kubectl describe pod` and `kubectl logs`; add `--previous` for the last crashed instance. Do not paste credentials or user payloads into reports. Container statistics, guest memory and JVM heap have different scopes: they should not be expected to show the same number.

5.3 From an image to a reachable application

```
python3 microservices/tools/workloads.py up
python3 microservices/tools/workloads.py status
python3 microservices/tools/workloads.py forward
# Open http://localhost:18080
python3 microservices/tools/workloads.py tunnel-stop
```

The forward binds only `127.0.0.1:18080` and reaches the guest's frontend NodePort 30080. The canonical optimized frontend is packaged with the existing production Dockerfile. Its nginx configuration routes API prefixes to the appropriate private Kubernetes Service and refuses internal/management paths. There is no second Java gateway JVM. The gateway layer performs HTTP routing; service authentication and business authorization remain separate responsibilities.

5.4 Helm: application packaging inside Kubernetes

A chart is a directory of reusable Kubernetes templates plus values. A release is an installed chart instance with a revision history. Here, `deploy/chart` is the chart and `myscoutee-msc121` is its release and namespace. Terraform owns the virtual machine, network and disks; Helm owns the application resources inside the cluster. Neither compiles Java or builds images during startup.

`workloads.py` supplies the assembly map, locked images, runtime values and private generated credentials to Helm through standard input. It reuses the canonical Mongo initialization and nginx response policy. It verifies namespace identity and ownership before adopting this task's earlier manifests; it refuses foreign releases. Helm client-side patching avoids conflicts with the earlier lab apply manager. An unsuccessful readiness wait returns a failure and leaves resources available for diagnosis.

```
python3 microservices/tools/workloads.py chart-check
microservices/tools/helm.sh list -n myscoutee-msc121
microservices/tools/helm.sh status myscoutee-msc121 -n myscoutee-msc121
microservices/tools/helm.sh history myscoutee-msc121 -n myscoutee-msc121
```

`helm.sh` downloads the pinned, checksum-verified tool into private lab state. It keeps its configuration/cache and kubeconfig out of the user's default directories. Helm values and release Secrets contain credentials; do not publish `helm get values` or manifest output. Mongo's volume claim is retained during release removal; destroying the VM still deletes the lab disk.

The completed `helm-install-gateway-api-crd` system Job belongs to k3s. It installs Kubernetes Gateway API resource definitions, not a running API gateway. Our actual HTTP entry layer is nginx. CoreDNS resolves Service names such as `mongodb` inside the cluster; `local-path-provisioner` allocates a directory on the guest disk for Mongo's claim. Neither is one of the eight Java business services.

6. Prometheus, Grafana and Localhost Access

6.1 Collection, storage and display

The measurement path is **Spring/Micrometer → private HTTP metrics endpoint → Prometheus → Grafana → browser**. Micrometer exposes measurements from each JVM. Prometheus periodically reads those endpoints, records time series and evaluates queries.

Grafana's provisioned datasource points to Prometheus and its panels display query results. Grafana does not collect JVM measurements itself.

In this implementation, `render-deployment.py` generates Prometheus targets from `assemblies.json`. Scrapes run every 30 seconds with a five-second timeout. The Grafana dashboard also refreshes every 30 seconds. Its datasource uses `http://127.0.0.1:19090`. Both monitoring containers use host networking with explicit loopback-only listeners; this lets Prometheus reach the libvirt guest without Docker bridge forwarding or changes to host firewall rules.

6.2 Start and open the monitoring tools

```
microservices/tools/lab.sh monitoring-up
microservices/tools/lab.sh monitoring-stop
microservices/tools/lab.sh monitoring-destroy
```

The separately testable monitoring stack binds Prometheus to `127.0.0.1:19090` and Grafana to `127.0.0.1:13000`. Grafana uses account `lab`; the loopback-only lab default password is `msc121-local-only`. The private `monitoring.env` file retains the configured credential. Provisioned dashboards cover availability, JVM heap, request rate, latency, CPU and errors.

Browser address	What to inspect
localhost:19090/targets	Configured scrape targets, their last scrape and errors
localhost:19090/query	PromQL expressions such as `up`
localhost:13000/d/myscoutee-services	Sign in and view the provisioned MyScoutee service dashboard

The default above is a public convenience credential for this local lab. An existing custom password is preserved; read that value locally from the private state file. Repeated startup preserves the credential and existing monitoring volumes. The startup helper waits for Prometheus readiness before requesting a configuration reload. Monitoring containers have no restart-at-boot policy.

Prometheus and Grafana each have a 256 MiB container limit. Metrics retention is bounded to 24 hours / 256 MB. Plugin installation and runtime updates are disabled. Services publish health and metrics on private port 8081, separated from business port 8080. Public ingress must not expose that management port.

6.3 Interpret the provisioned dashboard

Panel	Interpretation and limit
Scrape availability	`up=1` means the metrics scrape succeeded. `up=0` means it failed; inspect the target error. This does not test login, Mongo writes or an entire user flow.
JVM heap used	Managed Java heap in bytes, grouped by service. Native buffers, threads, metaspace and other processes require separate memory observations.

Panel	Interpretation and limit
HTTP requests per second	A five-minute rate of the request counter. A quiet system can have a valid near-zero rate. Missing samples are different from zero traffic.
HTTP p95 latency	A histogram estimate of the duration below which approximately 95% of observed requests fall. It needs request samples and histogram buckets; a blank panel before traffic is not a latency result.
JVM process CPU	Reported process CPU usage. Compare with guest/runtime statistics and the configured CPU constraints before drawing capacity conclusions.
HTTP server errors per second	The rate of 5xx responses. It omits 4xx outcomes and application-level failures returned with successful HTTP status, which the flow/stress assertions must catch.

Start with `up{job="prometheus"}` in Prometheus: it checks the collector's own scrape. Then inspect `up{job="myscoutee"}` by service. Check private gateway connectivity from the Prometheus container when application targets cannot be scraped. A stopped or unreachable backend makes its target DOWN; missing JVM series and missing HTTP traffic are distinct from scrape failure. Use the per-service labels and the VM/container statistics alongside JVM panels.

> **Two different DNS and network scopes**
> Kubernetes Service DNS is internal to the guest cluster. Prometheus therefore scrapes `192.168.173.10:30081/metrics/<service>`; a separate nginx listener maps only the generated service names to their private `8081/metrics` endpoint. Its resolver uses fully qualified Kubernetes names. Other paths return 404, and the public frontend listener does not expose this route. Targets and service labels are generated from the same assembly manifest.

The checked-in dashboard and datasource provisioning are the durable configuration. Record intended edits in those files, then restart/reconcile the owned monitoring stack; browser-only changes are not a reproducible setup. `monitoring-stop` preserves samples and Grafana data; `monitoring-destroy` deletes the owned volumes and credential so the next startup is a fresh monitoring installation.

6.4 Application access and diagnosis sequence

> **Application browser entry point — completion requirement**
> The production-built frontend is available at localhost:18080 through the versioned, loopback-only SSH forward. nginx routes API paths to the owning Kubernetes Services. Grafana availability is not application availability.

Find the first failing layer

Observation	Next check
VM absent or stopped	`vm.py status`, host service state, then explicit lifecycle startup. Do not delete state to bypass an ownership failure.

Observation	Next check
VM running, Kubernetes unavailable	`vm.py ssh` reports cloud-init and k3s status; use the guest console for systemd diagnostics.
Node Ready, pod missing or restarting	Deployment counts, pod events, image availability and container logs. A built JAR does not supply missing remote beans.
Grafana unavailable, Prometheus ready	Owned Grafana container health and datasource connection. Recreating the VM cannot fix a host Grafana problem.
Grafana opens, application metrics absent	Prometheus target status, private endpoint and network/DNS path, then time range and actual traffic.
Metrics healthy, browser/API flow fails	Public routing, authentication, remote adapters and data behavior for the exact failing request. Collector availability is insufficient.

Tracing and durable-operation metrics remain to be added alongside the corresponding boundaries. Keep user, workspace, session and payment identifiers out of metric labels. An application target correctly remains DOWN while its deployment is absent.

7. Bounded Stress and Further Qualification

The bounded load harness uses `microservices/tools/check-load.py` against six actual authenticated application routes, plus the separate two-pod contact-write harness. These reuse the lab’s native session and deployment helpers. For broader business stress campaigns, use the existing `e2e/stress/backend` runner and scenarios with the microservices localhost ingress; do not start its monolith Compose example alongside the lab.

The following sequence also applies when extending the measured flow set:

1. Complete the remote bindings and authenticated execution context required by the selected full-application flows.
2. Deploy production-style application images, one shared Mongo replica set, required Redis/object storage and bounded monitoring into the owned lab.
3. Prepare isolated canonical fixtures once, exit the preparation process, then verify REAL/DEMO and workspace behavior through the public API.
4. Open the localhost application for the user. Preserve the repository's manual-QA/browser-control boundary unless full-flow browser testing is explicitly delegated.
5. Run a recorded contract/flow smoke test, then the applicable stress scenarios and bounded load ramp. Do not interpret failed business assertions as acceptable performance.
6. Record exact image identities, fixture identity, concurrency, resource limits, error rate, throughput, p50/p95/p99 latency, CPU, memory and database behavior.
7. Compare against an equivalent monolith baseline serially on this machine. Keep build time and fixture preparation outside timed results.

The read-load harness can temporarily add a second assets replica. It records request failures, route latency and resource samples, then revokes its session and restores the replica count. Keep reports with the associated QA evidence; a bounded read run does not establish write saturation

or production exhaustion limits.

```
python3 microservices/tools/check-load.py --two-assets --seconds 60 --concurrency 8
```

Same-scenario monolith comparison

Compare deployments serially with the same source revisions, workload, data, concurrency, warmup and resource limits. Record increases as well as decreases in latency, throughput and resource use. A historical result with different host load, cache state or fixtures cannot isolate an architecture regression.

The existing `server-events-read` and `server-read-sweep` profiles reuse the same assertions behind either architecture. Native DEMO sessions and canonical lab data differ from an isolated monolith E2E seed; record that difference when comparing deployed systems. Functional success alone does not establish performance parity.

8. Stop, Destroy and Host Cleanup

Operation	Meaning
`vm.py stop`	Gracefully shut down the lab and deactivate its network/pool; retain disks and identity. No forced power loss on timeout.
`vm.py destroy`	Explicitly delete Terraform-owned VM/network/pool/disks, then remove guest credentials, image staging and state after confirmed removal.
`host.py stop`	Stop owned host services after the guest is off. Preserve foreign resources and pre-existing services.
`host.py purge`	Remove tracked added host packages and membership only after all lab resources are destroyed.

```
# Full owned cleanup, including host additions (deletes lab data):
bash microservices/tools/cleanup-lab.sh --purge-host

# Or select operations individually:
python3 microservices/tools/vm.py stop
# Retain the lab for the next explicit start, or remove it:
python3 microservices/tools/vm.py destroy
sudo python3 microservices/tools/host.py purge
microservices/tools/terraform.sh clean-tools

microservices/tools/lab.sh monitoring-destroy
microservices/tools/lab.sh clean-images
microservices/tools/lab.sh clean-build
```

Destroy is intentionally different from stop: it deletes lab data. Host purge refuses changed package versions, foreign libvirt resources and package-removal plans affecting unowned packages. There is no global package autoremove, Docker prune, firewall flush or broad process kill. Tiny lifecycle lock files may remain to keep concurrent command locking safe.

Image cleanup uses recorded tags and exact IDs; changed tags are preserved. Build cleanup refuses to remove configuration mounted by active monitoring. Keep the state required for recovery after interrupted operations; never delete Terraform state as a shortcut to a fresh start.

9. Rules for Future Changes

The trust kernel routes backend domain/dependency/configuration/worker/transport/deployment changes to `.agents/trust/reviews/microservices-compatibility.md`. New agents must preserve both deployment forms.

- Keep cross-domain dependencies on API/DTO contracts, with one implementation owner. Avoid foreign services, repositories and documents.
- Make domain Spring configuration usable in the monolith and selected assemblies; keep shell-only imports out of canonical code.
- Review network cost when an interface moves across processes. Batch hot reads and avoid remote calls inside per-item loops.
- Keep schedulers, indexes and durable side effects under explicit ownership. Replica count must not create duplicate work.
- Propagate validated identity and scope, restore contexts on errors, and treat remote timeout as an unknown outcome for mutations.
- Extend the actual owner of compensation, retry or outbox behavior. Introduce saga infrastructure only for a proven missing capability.
- Keep every installation and cleanup step versioned and idempotent. Secrets, state and VM disks stay outside Git.

10. Verification and Known Limits

Verification layer	Boundary
Assembly audit	Domain coverage and exclusion of foreign owner implementations
Canonical and adapter tests	Business semantics, detached contracts, execution context and failure handling
Infrastructure checks	Deployment ownership, upgrade guards and cleanup behavior
Runtime readiness	Wiring and startup; does not prove complete business-flow correctness
Public API and browser checks	Authentication, routing and the selected user flows
Replica and recovery checks	Selected write, replay and worker-failure behavior
Bounded load and metrics	Measured workload behavior; does not establish production capacity

Keep execution results, source and image identities, fixture details and outstanding checks in the relevant QA records. This guide describes the verification tools and their limits.

```
microservices/tools/lab.sh jars
node microservices/tools/verify-transport-contracts.mjs
python3 microservices/tools/test_infra.py
microservices/tools/terraform.sh validate
node guides/manuals/build-manuals.mjs MSC-MS-001
```

10.1 Mutation recovery and sampled replica writes

The domain owners retain contact commands, profile status/support work, event membership and feedback counter receipts, and asset/event lifecycle delivery. These reuse the native notification outbox, scheduled-message tasks and affinity queue. Stable lifecycle occurrence IDs prevent a repeated request from restarting a completed email task; revision-guarded attachment projections prevent a late trash update from undoing a restore. Embedded tests cover lost replies, replay, restart, leases and stale writes.

```
python3 microservices/tools/check-business.py --two-identity
```

The contact-write harness uses two disposable DEMO actors for contact request, reject, approve, concurrent requests and repeated approvals. It checks durable command and notification cardinalities and per-pod successful write counters. Only identity gains a second replica, subject to the memory gate. Cleanup revokes its sessions, restores one replica and removes only receipt-owned data. If interrupted, use the printed `--cleanup RUN_ID` command. It does not reset canonical users, mutate REAL or qualify payments.

The `check-business.py --worker-recovery --write-cycles 20` mode temporarily rejects contact delivery through an owned proxy, terminates the identity worker after journal acceptance, and checks completion after routing restoration without API replay. Cleanup restores temporary routing and removes owned sessions and fixtures.

Managed browser qualification

The versioned browser-session launcher and `microservices/tools/check-browser.mjs` verify the production HTTP path: landing consent and carousel controls, native DEMO login, game route, event/explore popups, Work/Dating workspace switches and native logout. The check rejects a local-adapter fallback. Screenshots and a credential-free response/status report are retained under `microservices/build/reports/browser`. This automated flow requires explicit user delegation; ordinary manual-QA control remains with the user.

10.2 Coordinated writer upgrade

The `assets`, `events` and `feedbackStates` owners initialize missing document versions before accepting writes. All old backend writers must be stopped before running new contexts or deploying new images. The lab deployment guard refuses a changed backend image while an old backend pod remains. Supporting services and volumes stay available.

```
python3 microservices/tools/workloads.py backends-stop
python3 microservices/tools/check-contexts.py
# Build and lock images explicitly before deployment; up never builds.
python3 microservices/tools/workloads.py up
```

A monolith upgrade needs the same stop-old/start-new boundary. Rollback to a pre-version writer requires a matched database backup, not merely an old image. Integration image imports retain deterministic partial objects after an unknown result; automatic orphan garbage collection is not included. Provider delivery remains at least once. Publication's existing invitation, reminder, chat and tournament flows still require full-flow qualification; the lifecycle receipt alone does not prove those adjacent effects.

10.3 Background CPU and repeated owner calls

Inspect private request metrics together with per-thread and container CPU samples to distinguish incoming owner calls from local polling. Include startup, JIT and periodic jobs when interpreting short samples.

The shared scheduled-group orchestration calls the existing atomic scheduled claim, which checks enabled state, due time and ownership. Affinity layout likewise uses one owner call for the claim instead of separate find, due and claim calls. Preserve missing, disabled and not-due rejection, acknowledged-claim execution and exclusion of another run when changing these paths.

Batch owner reads and reuse atomic owner operations where their existing semantics fit. Fewer remote requests can reduce work, but a sustained CPU or latency improvement requires a matched measurement.

10.4 References for the tools actually used

These upstream references explain the concepts behind this guide. The repository's pinned versions and scripts remain the authority for this lab's exact behavior; upstream defaults may differ.

- libvirt QEMU/KVM driver — hypervisor access and the system connection.
- Terraform state — the binding between declarations and real resource identities.
- k3s architecture and bundled CLI tools — server/worker placement and inspection.
- Kubernetes Pods and Services — workload and network identities.
- Prometheus overview — collection and time-series queries.
- Grafana introduction — querying and visualizing datasource results.

Before production adoption, qualify the required browser and business flows, worker and shared Mongo failure recovery, fresh installation, host cleanup and representative write load. Record results and limitations in QA documentation. The architecture plan is `guides/plans/MICROSERVICES_ARCHITECTURE.md`; operational instructions are maintained in `microservices/README.md`.